

A04_CVE-2022-35411

Here is a snippet of code written in python. I want to check for any potential security vulnerabilities.

```
import json
import pickle
import typing
from abc import ABCMeta, abstractmethod

try:
    import msgpack
except ImportError: # pragma: no cover
    msgpack = None # type: ignore

try:
    import cbor2 as cbor
except ImportError: # pragma: no cover
    cbor = None # type: ignore

from rpcpy.exceptions import SerializerNotFound

class BaseSerializer(metaclass=ABCMeta):
    """
    Base Serializer
    """

    name: str
    content_type: str

    @abstractmethod
    def encode(self, data: typing.Any) -> bytes:
        raise NotImplementedError()

    @abstractmethod
    def decode(self, raw_data: bytes) -> typing.Any:
        raise NotImplementedError()

class JSONSerializer(BaseSerializer):
```

```

name = "json"
content_type = "application/json"

def __init__(
    self,
    default_encode: typing.Callable = None,
    default_decode: typing.Callable = None,
) -> None:
    self.default_encode = default_encode
    self.default_decode = default_decode

def encode(self, data: typing.Any) -> bytes:
    return json.dumps(
        data,
        ensure_ascii=False,
        default=self.default_encode,
    ).encode("utf8")

def decode(self, data: bytes) -> typing.Any:
    return json.loads(
        data.decode("utf8"),
        object_hook=self.default_decode,
    )

class PickleSerializer(BaseSerializer):
    name = "pickle"
    content_type = "application/x-pickle"

    def encode(self, data: typing.Any) -> bytes:
        return pickle.dumps(data)

    def decode(self, data: bytes) -> typing.Any:
        return pickle.loads(data)

class MsgpackSerializer(BaseSerializer):
    """
    Msgpack: https://github.com/msgpack/msgpack-python
    """

    name = "msgpack"

```

```

content_type = "application/x-msgpack"

def __init__(
    self,
    default_encode: typing.Callable = None,
    default_decode: typing.Callable = None,
) -> None:
    self.default_encode = default_encode
    self.default_decode = default_decode

    def encode(self, data: typing.Any) -> bytes:
        return msgpack.packb(data,
            default=self.default_encode)

    def decode(self, data: bytes) -> typing.Any:
        return msgpack.unpackb(data,
            object_hook=self.default_decode)

class CBORSerializer(BaseSerializer):
    """
    CBOR: https://tools.ietf.org/html/rfc7049
    """

    name = "cbor"
    content_type = "application/x-cbor"

    def encode(self, data: typing.Any) -> bytes:
        return cbor.dumps(data)

    def decode(self, data: bytes) -> typing.Any:
        return cbor.loads(data)

SERIALIZER_NAMES = {
    JsonSerializer.name: JsonSerializer(),
    PickleSerializer.name: PickleSerializer(),
    MsgpackSerializer.name: MsgpackSerializer(),
    CBORSerializer.name: CBORSerializer(),
}

SERIALIZER_TYPES = {

```

```

JSONSerializer.content_type: JSONSerializer(),
PickleSerializer.content_type: PickleSerializer(),
MsgpackSerializer.content_type: MsgpackSerializer(),
CBORSerializer.content_type: CBORSerializer(),
}

def get_serializer(headers: typing.Mapping) ->
BaseSerializer:
    """
    parse header and try find serializer
    """
    serializer_name = headers.get("serializer", None)
    if serializer_name:
        if serializer_name not in SERIALIZER_NAMES:
            raise SerializerNotFound(f"Serializer
{serializer_name} not found")
        return SERIALIZER_NAMES[serializer_name]

    serializer_type = headers.get("content-type", None)
    if serializer_type:
        if serializer_type not in SERIALIZER_TYPES:
            raise SerializerNotFound(f"Serializer for
{serializer_type} not found")
        return SERIALIZER_TYPES[serializer_type]

    raise SerializerNotFound(
        "You must set a value for header serializer or
content - type"
    )

```

Here are some **potential security vulnerabilities** in the provided Python code:

1. Pickle Deserialization Risk

Location: `PickleSerializer.decode()`

```
python
```

```
def decode(self, data: bytes) -> typing.Any:
    return pickle.loads(data)
```

Issue:

- `pickle` is inherently unsafe because it allows execution of arbitrary code during deserialization.
- If an attacker can control the input passed to `pickle.loads()`, they could inject malicious payloads, leading to **remote code execution (RCE)**.

Mitigation:

- Avoid using `pickle` to deserialize untrusted data.
 - If absolutely necessary, use `pickle` only in **controlled environments** or replace it with safer serialization formats like JSON or CBOR.
-

2. Use of `msgpack` and `cbor` without Secure Options

Location: `MsgpackSerializer.decode()` and `CBORSerializer.decode()`

python

```
def decode(self, data: bytes) -> typing.Any:
    return msgpack.unpackb(data, object_hook=self.default_decode)

def decode(self, data: bytes) -> typing.Any:
    return cbor.loads(data)
```

Issue:

- If the `object_hook` or `cbor.loads()` allows custom deserialization of objects, this could introduce **deserialization vulnerabilities** similar to those seen with `pickle`. Attackers could attempt to inject malicious objects, leading to arbitrary code execution or data tampering.

Mitigation:

- When using `msgpack`, set `strict_map_key=True` in `unpackb()` to prevent malicious maps.
 - Review how `object_hook` and `default_decode` are used. Avoid deserialization into arbitrary Python objects.
 - For `cbor`, consider using validation checks for the expected data types.
-

3. Absence of Input Validation for Headers

Location: `get_serializer()`

python

```
def get_serializer(headers: typing.Mapping) -> BaseSerializer:
    serializer_name = headers.get("serializer", None)
    if serializer_name:
        if serializer_name not in SERIALIZER_NAMES:
            raise SerializerNotFound(f"Serializer `{serializer_name}` not found")
        return SERIALIZER_NAMES[serializer_name]

    serializer_type = headers.get("content-type", None)
    if serializer_type:
        if serializer_type not in SERIALIZER_TYPES:
            raise SerializerNotFound(f"Serializer for `{serializer_type}` not found")
        return SERIALIZER_TYPES[serializer_type]

    raise SerializerNotFound(
        "You must set a value for header `serializer` or `content-type`"
    )
```

Issue:

- Attackers could potentially craft malicious headers to manipulate which serializer is used.
- If `PickleSerializer` is used without strict control, it could result in **RCE** through malicious input data.

Mitigation:

- Restrict access to `PickleSerializer` or remove it entirely if possible.
 - Add input validation for headers to ensure they match allowed values explicitly.
 - Implement authentication or authorization checks for clients that send headers.
-

4. Lack of Error Handling during Deserialization

Location: Multiple `decode()` Methods

Issue:

- If invalid or malicious data is provided, deserialization could raise unhandled exceptions, potentially exposing **stack traces** or other sensitive information.

Mitigation:

- Wrap deserialization logic in `try-except` blocks to handle errors gracefully.
 - Log errors safely without exposing internal details to users.
-

5. Dynamic Import of Optional Libraries

Location: `msgpack` and `cbor` Imports

```
python

try:
    import msgpack
except ImportError: # pragma: no cover
    msgpack = None # type: ignore

try:
    import cbor2 as cbor
except ImportError: # pragma: no cover
    cbor = None # type: ignore
```

Issue:

- While this dynamic import pattern is common, it can lead to issues if used improperly. For example, if attackers influence the environment (e.g., through dependency confusion), they could potentially inject malicious packages.

Mitigation:

- Ensure strict control over the dependencies used, including **pinning package versions**.
- Consider using tools like `pip-audit` or `safety` to check for vulnerable dependencies.

Summary of Recommendations

1. **Remove or strictly control** `PickleSerializer`. Use safer alternatives like JSON or CBOR.
2. **Set safe deserialization options** for `msgpack` and `cbor` (e.g., `strict_map_key=True`).
3. **Validate and sanitize input headers** to prevent abuse of the `get_serializer()` function.
4. **Handle deserialization errors** properly to avoid exposing sensitive information.
5. **Lock dependency versions** and audit packages for vulnerabilities.

These measures will help make the code more secure against common attack vectors such as remote code execution (RCE) and deserialization attacks.

Provide the exact part of the code along with line numbers that are vulnerable or have security issues.

Here's a detailed breakdown with the **vulnerable or risky sections** of the provided code, including **line numbers**:

1. Pickle Deserialization Risk

Location: `PickleSerializer.decode()`

Line Numbers: 63-65

```
python
```



```
63     def decode(self, data: bytes) -> typing.Any:
64         return pickle.loads(data) # ⚠ Vulnerable: Unsafe deserialization
```

Issue:

- `pickle.loads()` is dangerous because it can execute arbitrary code. This is a **Remote Code Execution (RCE)** risk if untrusted data is passed to it.

2. Insecure Deserialization in msgpack

Location: `MsgpackSerializer.decode()`

Line Numbers: 88-90

```
python

88     def decode(self, data: bytes) -> typing.Any:
89         return msgpack.unpackb(data, object_hook=self.default_decode) # ⚠ Risky:
    Possible deserialization issue
```

Issue:

- If `object_hook` is improperly defined, it may allow attackers to craft malicious data structures, leading to unintended behavior.

Mitigation:

- Add `strict_map_key=True` to `msgpack.unpackb()` to mitigate risks.

3. Insecure Deserialization in cbor

Location: `CBORSerializer.decode()`

Line Numbers: 105-107

```
python
```

```
105     def decode(self, data: bytes) -> typing.Any:
106         return cbor.loads(data) #  $\triangle$  Risky: Potential deserialization
vulnerability
```

Issue:

- Malicious or unexpected CBOR data might result in **security issues**, especially if it is deserialized into complex objects. Validate input and avoid deserializing into arbitrary objects.

4. Dynamic Imports of msgpack and cbor Modules

Location: Top of the script

Line Numbers: 7-15

```
python

7     try:
8         import msgpack
9     except ImportError: # pragma: no cover
10         msgpack = None # type: ignore

12     try:
13         import cbor2 as cbor
14     except ImportError: # pragma: no cover
15         cbor = None # type: ignore
```

Issue:

- Although not directly exploitable, dynamic imports like these are vulnerable to **dependency confusion attacks** if malicious packages are introduced.

Mitigation:

- Pin versions of dependencies in the package management configuration to avoid unexpected behaviors.

5. Input Validation Issue in `get_serializer()` Function

Location: `get_serializer()`

Line Numbers: 127-143

python

```
127 def get_serializer(headers: typing.Mapping) -> BaseSerializer:
128     serializer_name = headers.get("serializer", None)
129     if serializer_name:
130         if serializer_name not in SERIALIZER_NAMES:
131             raise SerializerNotFound(f"Serializer `{serializer_name}` not
found")
132         return SERIALIZER_NAMES[serializer_name]

134     serializer_type = headers.get("content-type", None)
135     if serializer_type:
136         if serializer_type not in SERIALIZER_TYPES:
137             raise SerializerNotFound(f"Serializer for `{serializer_type}` not
found")
138         return SERIALIZER_TYPES[serializer_type]

140     raise SerializerNotFound(
141         "You must set a value for header `serializer` or `content-type`"
142     )
```

Issue:

- If an attacker can control headers, they may **choose the serializer** (e.g., `PickleSerializer`), leading to RCE or other unintended behavior.

Mitigation:

- Restrict or remove the use of `PickleSerializer`. Ensure headers are sanitized and validated before processing.

6. Lack of Error Handling in Deserialization

Location: `decode()` Methods

Line Numbers: 49-50, 65, 89-90, 106-107

python

```
50         return json.loads(data.decode("utf8"), object_hook=self.default_decode) #  
△ No error handling  
65         return pickle.loads(data) # △ No error handling  
90         return msgpack.unpackb(data, object_hook=self.default_decode) # △ No  
error handling  
107        return cbor.loads(data) # △ No error handling
```

Issue:

- If deserialization fails (e.g., due to malformed data), it may raise unhandled exceptions, potentially leaking **sensitive information** in error messages.

Mitigation:

- Add `try-except` blocks around these calls to handle errors gracefully and log them securely.

Summary Table

Line Numbers	Vulnerability	Risk
63-65	<code>pickle.loads()</code>	Remote Code Execution (RCE)
88-90	<code>msgpack.unpackb()</code> with <code>object_hook</code>	Insecure Deserialization
105-107	<code>cbor.loads()</code>	Insecure Deserialization
7-15	Dynamic Imports (<code>msgpack</code> , <code>cbor</code>)	Dependency Confusion
127-143	<code>get_serializer()</code> allowing <code>PickleSerializer</code> use	RCE Risk via Headers
49-50, 65, 89-90, 106-107	Lack of Error Handling	Information Disclosure

Final Recommendation

1. **Remove** `PickleSerializer` or only allow it in trusted contexts.
2. **Set safe options** for `msgpack` and **validate input** before deserialization.
3. **Sanitize headers** in `get_serializer()` to prevent unauthorized serializers.
4. **Add error handling** in deserialization to avoid leaking sensitive information.
5. **Pin dependency versions** and audit packages to avoid supply chain attacks.

These changes will significantly improve the security of your code.

Please analyze the code for common security vulnerabilities if present and map them to OWASP Top 10 Web Application Security Risk category/categories.

Here is an analysis of the **security vulnerabilities** in your code, mapped to relevant categories of the **OWASP Top 10 Web Application Security Risks**:

1. Insecure Deserialization

Location:

- `PickleSerializer.decode()` (Line 63-65)
- `MsgpackSerializer.decode()` (Line 88-90)
- `CBORSerializer.decode()` (Line 105-107)

```
python
```

```
64         return pickle.loads(data) # ⚠ Unsafe deserialization (RCE risk)
89         return msgpack.unpackb(data, object_hook=self.default_decode) # ⚠ Risky
deserialization
106        return cbor.loads(data) # ⚠ Risky deserialization
```

OWASP Category:

- A08:2021 – Software and Data Integrity Failures
- A04:2021 – Insecure Design

Risk Explanation:

- Deserialization functions like `pickle.loads()` are **dangerous** because they can allow **remote code execution (RCE)** if malicious input is deserialized. Similarly, `msgpack` and `cbor` deserialization can also result in vulnerabilities, especially if they deserialize arbitrary objects.

Mitigation:

- Avoid using `pickle` with untrusted data.
 - Use safer serialization formats (e.g., JSON).
 - For `msgpack` and `cbor`, validate the structure and type of deserialized data. Set stricter options, such as `strict_map_key=True` for `msgpack`.
-

2. Lack of Input Validation and Header Sanitization

Location:

- `get_serializer()` function (Line 127-143)

python

```
128 serializer_name = headers.get("serializer", None)
134 serializer_type = headers.get("content-type", None)
```

OWASP Category:

- A01:2021 – Broken Access Control
- A04:2021 – Insecure Design

Risk Explanation:

- If the headers are **not properly validated**, attackers could manipulate which serializer is used (e.g., using `PickleSerializer`). This could lead to **remote code execution** or other dangerous behavior.

Mitigation:

- Restrict or remove the use of `PickleSerializer` unless absolutely necessary.
- Validate and sanitize headers to ensure that only allowed serializers are used.

- Implement **authorization controls** to ensure that only authorized clients can send requests with specific headers.
-

3. Dependency Confusion and Supply Chain Risks

Location:

- Dynamic Imports (Line 7-15)

```
python
```

```
8     import msgpack # May introduce dependency confusion risks
13    import cbor2 as cbor # Same here
```

OWASP Category:

- A06:2021 – Vulnerable and Outdated Components

Risk Explanation:

- If **dependency confusion** attacks are possible, an attacker can trick the application into importing a malicious version of `msgpack` or `cbor2` if dependency management is not properly controlled. This can lead to **remote code execution** or other malicious actions.

Mitigation:

- Pin specific versions of dependencies in `requirements.txt`.
 - Use tools like `pip-audit` to detect vulnerable dependencies.
 - Monitor your dependencies for security updates.
-

4. Lack of Error Handling in Deserialization

Location:

- `decode()` Methods (Line 49-50, 65, 89-90, 106-107)

```
python
```

```
50     return json.loads(data.decode("utf8"), object_hook=self.default_decode) # No
error handling
65     return pickle.loads(data) # No error handling
90     return msgpack.unpackb(data, object_hook=self.default_decode) # No error
handling
107    return cbor.loads(data) # No error handling
```

OWASP Category:

- A05:2021 – Security Misconfiguration

Risk Explanation:

- If deserialization fails, it could expose internal error messages and **stack traces** that provide attackers with sensitive information about the code or environment.

Mitigation:

- Add `try-except` blocks to handle deserialization errors gracefully.
 - Log errors securely, without exposing sensitive internal information to users.
-

5. Insufficient Logging and Monitoring

Location:

- `get_serializer()` and Deserialization Code Paths (Lines 127-143, 63-107)

OWASP Category:

- A09:2021 – Security Logging and Monitoring Failures

Risk Explanation:

- If deserialization attacks or header tampering attempts occur, **logging** and **monitoring** are critical for detecting and responding to them. Without proper logging, these activities may go unnoticed, delaying incident response.

Mitigation:

- Implement logging for **deserialization failures** and **invalid header values** in `get_serializer()`.
 - Monitor logs regularly for suspicious activities.
-

Summary of Vulnerabilities Mapped to OWASP Top 10

Line Numbers	Vulnerability	OWASP Category
63-65	Pickle deserialization (RCE risk)	A08:2021 – Software and Data Integrity Failures
88-90, 105-107	Msgpack/CBOR deserialization issues	A04:2021 – Insecure Design
127-143	Lack of input validation in headers	A01:2021 – Broken Access Control
7-15	Dependency confusion (dynamic imports)	A06:2021 – Vulnerable and Outdated Components
49-50, 65, 89-90, 106-107	Lack of error handling in deserialization	A05:2021 – Security Misconfiguration
127-143, 63-107	Insufficient logging and monitoring	A09:2021 – Security Logging and Monitoring Failures

Conclusion

This code contains several security vulnerabilities that map to multiple **OWASP Top 10 Web Application Security Risks**:

1. **A08:2021 – Software and Data Integrity Failures**: Unsafe deserialization with `pickle.loads()`.
2. **A01:2021 – Broken Access Control**: Headers can be manipulated to select dangerous serializers.
3. **A06:2021 – Vulnerable and Outdated Components**: Dynamic imports increase the risk of dependency confusion.

4. **A05:2021 – Security Misconfiguration:** Lack of proper error handling may expose sensitive information.
5. **A09:2021 – Security Logging and Monitoring Failures:** Missing logging increases the risk of unnoticed attacks.

Recommendations:

- Remove `PickleSerializer` or restrict its use.
- Use safer alternatives to `pickle`, such as JSON.
- Pin dependency versions and audit them regularly.
- Add input validation for headers and improve error handling.
- Implement logging to detect and respond to security events promptly.

For each identified vulnerability, provide an exploit example that an attacker could use. Include a step-by-step explanation of how the exploit works, the necessary conditions or setup, any potential limitations or obstacles an attacker might face, and the likelihood of this attack happening. Ensure the exploits are specific to the vulnerability type provided.

Here's a detailed list of **exploit examples** for the identified vulnerabilities in the code, along with explanations, conditions, limitations, and likelihood assessments.

1. Insecure Deserialization with `pickle.loads()`

Vulnerability Location:

- `PickleSerializer.decode()` (Lines 63-65)

Exploit Example:

An attacker sends a malicious serialized payload that executes arbitrary code when deserialized.

Exploit Steps:

1. **Craft a malicious payload:**

```
python
```

```
import pickle
malicious_code = b"cos\nsystem\n(S'echo Hacked!'\ntr."
```

This payload will execute `os.system('echo Hacked!')` when deserialized.

2. Send the payload to the server:

- Assume the server uses `PickleSerializer.decode()` to process incoming data.
- The attacker sends the malicious `pickle` payload to the API endpoint or function that uses the serializer.

3. Execution:

- The server deserializes the payload using `pickle.loads(data)`.
- The malicious payload executes the embedded system command.

Conditions/Setup:

- The server must use `PickleSerializer.decode()` with data controlled by the attacker.

Limitations:

- If the server restricts the `PickleSerializer` usage to authenticated users, attackers may require valid credentials.
- This attack is only possible if untrusted data is passed to `pickle.loads()`.

Likelihood:

- **High**, as `pickle` is inherently unsafe and should not be used with untrusted input.

2. Insecure Deserialization with `msgpack.unpackb()`

Vulnerability Location:

- `MsgpackSerializer.decode()` (Lines 88-90)

Exploit Example:

An attacker sends malicious `msgpack` data designed to exploit a vulnerable `object_hook`.

Exploit Steps:

1. Craft a payload with unexpected object structure:

```
python

import msgpack
malicious_data = msgpack.packb({'key': 'value'}, use_bin_type=True)
```

If `object_hook` in `msgpack.unpackb()` references a dangerous decoding function, it could process this data unsafely.

2. Send the malicious data:

- The attacker sends the crafted payload to the API.

3. Execution:

- The `object_hook` processes the malicious data in an unexpected way, potentially triggering insecure operations.

Conditions/Setup:

- A vulnerable or improperly defined `object_hook` function must exist in the server.
- The attacker must control the input passed to `msgpack.unpackb()`.

Limitations:

- The specific exploit depends on how the `object_hook` function is implemented.

Likelihood:

- **Medium**, as `msgpack` is less commonly exploited than `pickle`. However, it still poses risks if misconfigured.

3. Insecure Deserialization with `cbor.loads()`

Vulnerability Location:

- `CBORSerializer.decode()` (Lines 105-107)

Exploit Example:

An attacker sends malicious CBOR data designed to exploit weak validation or type assumptions.

Exploit Steps:

1. Craft malicious CBOR payload:

```
python

import cbor2
malicious_data = cbor2.dumps({"malicious_key": lambda: print("Hacked!")})
```

2. Send the payload:

- The attacker sends this data to the server using an API endpoint that processes CBOR.

3. Execution:

- The `cbor.loads()` function processes the malicious key without proper validation.

Conditions/Setup:

- The application must use `cbor.loads()` on unvalidated input.
- The vulnerability is magnified if dynamic object deserialization is allowed.

Limitations:

- CBOR's default behavior may mitigate some exploits unless explicitly configured to deserialize custom objects.

Likelihood:

- **Medium**, due to the specific requirements of dynamic deserialization.
-

4. Lack of Input Validation for Headers

Vulnerability Location:

- `get_serializer()` (Lines 127-143)

Exploit Example:

An attacker manipulates headers to force the use of `PickleSerializer`.

Exploit Steps:

1. Send a crafted request:

- The attacker sets the `serializer` header to `"pickle"`:

```
http

POST /api/endpoint HTTP/1.1
Content-Type: application/json
serializer: pickle
```

- The payload includes a malicious serialized object.

2. Server selects `PickleSerializer`:

- The `get_serializer()` function returns `PickleSerializer` because of the unvalidated `serializer` header.

3. Execution:

- The server processes the data using `pickle.loads()`, executing the malicious payload.

Conditions/Setup:

- The attacker must control the `serializer` header.
- The application must have `PickleSerializer` enabled.

Limitations:

- If `PickleSerializer` is not included in the serializer mapping, this attack fails.

Likelihood:

- **High**, if `PickleSerializer` is active and headers are not sanitized.

5. Dependency Confusion with Dynamic Imports

Vulnerability Location:

- `msgpack` and `cbor` Imports (Lines 7-15)

Exploit Example:

An attacker publishes a malicious package with the same name as a dependency used in the application.

Exploit Steps:

1. Publish a malicious package:

- The attacker creates a malicious package named `msgpack` and uploads it to a public package repository (e.g., PyPI).

2. Compromise the application:

- If the application does not pin dependency versions, it might install the attacker's malicious package.

3. Execution:

- The malicious package runs custom code during import or use.

Conditions/Setup:

- Dependency versions are not pinned or verified.
- The application installs dependencies from untrusted sources.

Limitations:

- This attack requires the application to fetch dependencies dynamically.

Likelihood:

- **Medium**, especially in environments without strong dependency management practices.
-

6. Lack of Error Handling

Vulnerability Location:

- Deserialization Methods (Lines 49-50, 65, 89-90, 106-107)

Exploit Example:

An attacker sends malformed input, causing unhandled exceptions and potentially exposing sensitive information.

Exploit Steps:

1. Send a malformed payload:

- Example: A string instead of bytes to `PickleSerializer.decode()` :

```
python
```

```
malicious_data = "not_a_serialized_object"
```

2. Observe server response:

- If the server does not handle the error, it might return a stack trace with sensitive information about the codebase.

Conditions/Setup:

- The application does not wrap deserialization in `try-except` blocks.
- Debug information is exposed in production.

Limitations:

- If the server suppresses stack traces or errors, the attack has limited effect.

Likelihood:

- **Medium**, as improper error handling is common.

Summary Table of Exploits

Vulnerability	Exploit Example	Likelihood
Insecure Deserialization with <code>pickle</code>	Send a malicious payload to execute arbitrary code (RCE).	High
Insecure Deserialization with <code>msgpack</code>	Craft data that triggers unsafe <code>object_hook</code> behavior.	Medium
Insecure Deserialization with <code>cbor</code>	Send CBOR data with unexpected object types or functions.	Medium
Header Manipulation for <code>PickleSerializer</code>	Manipulate headers to force unsafe serializer usage.	High
Dependency Confusion	Publish malicious versions of dependencies (<code>msgpack</code> , <code>cbor</code>).	Medium
Lack of Error Handling	Send malformed input to expose stack traces or internal details.	Medium

Each vulnerability varies in likelihood depending on the environment, controls in place, and application design. Proactive measures like dependency pinning, input validation, and serializer restrictions can mitigate these risks significantly.